

## Solution Requirements

|               |                                                                |
|---------------|----------------------------------------------------------------|
| Date          | 26/06/2026                                                     |
| Team ID       | Not Assigned                                                   |
| Project Name  | Rising Waters: A Machine Learning Approach to Flood Prediction |
| Maximum Marks | 4 Marks                                                        |

### Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should do).
- Non-Functional Requirements (NFR) specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Fill out the descriptions and necessary details for each category provided below.

### Step 1: Functional Requirements (FR)

| S.No | Requirement Category   | Requirement Description                                                                                                                                                                                                                                            | Priority (High/Medium/Low) |
|------|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 1    | Authentication         | The system does not require user authentication. Any user can directly access the web application through a browser and submit weather data for flood prediction without logging in.                                                                               | Low                        |
| 2    | Authorization Levels   | No role-based authorization is implemented. All users (meteorologists, coordinators, analysts) have equal access to the home page, prediction form, and result pages.                                                                                              | Low                        |
| 3    | External Interfaces    | The system uses a Kaggle-sourced flood dataset (flood dataset.xlsx) for training. The trained XGBoost model (floods.save) and StandardScaler (transform.save) are loaded by the Flask app via the Joblib library.                                                  | High                       |
| 4    | Transaction Processing | When the user submits the prediction form via HTTP POST, the Flask backend receives the 10 weather parameters, scales them using transform.save, passes them to the XGBoost model, and returns a flood or no-flood prediction result to the appropriate HTML page. | High                       |
| 5    | Reporting              | The system displays the flood probability score and a binary prediction result (Flood / No Flood) on the result page (chance.html or no_chance.html), along with a summary of all input values used for the prediction.                                            | Medium                     |
| 6    | Business Rules         | If the XGBoost model predicts class 1 (Flood), the user is redirected to chance.html with flood advisory actions. If it predicts class 0 (No Flood), the user is redirected to no_chance.html with monitoring recommendations. All 10 input fields are mandatory.  | High                       |

| S.No | Requirement Category                     | Requirement Description                                                                                                                                                                                                                  | Priority (High/Medium/Low) |
|------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| 7    | <b>Compliance to Laws or Regulations</b> | The system uses publicly available open-source data from Kaggle. No personal user data is collected or stored. The application complies with general open-source data usage standards and does not violate any data privacy regulations. | Medium                     |
| 8    | <b>Other</b>                             | The system must be deployable on cloud platforms (Render.com) and accessible via a public URL. The model and scaler files must be version-controlled in the GitHub repository for reproducibility.                                       | High                       |

## Step 2: Non-Functional Requirements (NFR)

| S.No | NFR Category                                     | Requirement Description                                                                                                                                                                                   | Target Metric / Acceptance Criteria                                                    |
|------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 1    | <b>Performance &amp; Speed</b>                   | The Flask web application must respond to prediction requests quickly. The XGBoost model inference and data scaling should complete within acceptable time limits for real-time use.                      | Prediction result returned in < 2 seconds after form submission                        |
| 2    | <b>Scalability</b>                               | The system deployed on Render.com must handle multiple simultaneous prediction requests. The Flask application architecture supports horizontal scaling through Gunicorn workers.                         | Supports up to 100 concurrent users on free-tier deployment                            |
| 3    | <b>Security &amp; Data Privacy</b>               | No personal user data is collected or stored. All inputs are processed in memory and discarded after prediction. The application is served over HTTPS via Render.com.                                     | HTTPS enforced; no user data persisted; no database connections                        |
| 4    | <b>Reliability &amp; Availability</b>            | The application must be consistently available via the Render.com deployment. The model and scaler files must load correctly on every startup without errors.                                             | 99% uptime on Render free tier; auto-restart on failure                                |
| 5    | <b>Usability &amp; Accessibility</b>             | The web interface must be intuitive and usable by non-technical users such as meteorologists and coordinators. Input field labels must include valid data ranges as hints.                                | All 10 inputs clearly labelled with units and ranges; mobile-responsive design         |
| 6    | <b>Other (Maintainability &amp; Portability)</b> | The codebase must be modular and version-controlled via GitHub. The requirements.txt file must list all dependencies for easy environment reproduction. The system must run on Windows, Linux, and macOS. | All code on GitHub; pip install -r requirements.txt sets up environment in < 5 minutes |